Lab Manual: ETL with PySpark in a Databricks Notebook

Objective

By the end of this lab, you will:

- Create and configure a Databricks notebook.
- Use PySpark to read data from Databricks File System (DBFS) or Google Cloud Storage (GCS).
- Apply a simple transformation (filtering + column selection).
- Write the transformed results back in Parquet format.

Prerequisites

- Access to a Databricks workspace.
- An active Databricks cluster.
- Sample dataset uploaded to DBFS (`/FileStore/tables/sample_data.csv`).

(Tip: You can upload files through Databricks' UI: Data → Add Data → Upload File).

- (Optional) Access to a GCS bucket if you want to test external storage.

Step 1: Create and Configure Notebook

1. Log in to your Databricks workspace.
2. In the sidebar, click **New → Notebook**.
3. Set the name as: `PySpark_ETL_Demo`
4. Select **Python** as the default language.
5. Attach the notebook to your active cluster.

Step 2: Initialise Spark Session

In the first cell, add the following code:

```
from pyspark.sql import SparkSession

# Create or retrieve the Spark session
spark = SparkSession.builder.appName("PySpark_ETL_Demo").getOrCreate()
```

This sets up a Spark session named `PySpark_ETL_Demo` which manages all Spark operations in this notebook.

Step 3: Read Input Data

Option A: From DBFS

```
df = spark.read.csv(
    "/FileStore/tables/sample_data.csv",
    header=True,
    inferSchema=True
)
```

- `header=True` → Treats the first row as column names.
- `inferSchema=True` → Automatically detects column data types.

Option B: From Google Cloud Storage (Optional)

If you have GCS access configured, use:

```
# Example path: replace with your bucket name and file path
df = spark.read.csv(
    "gs://my-gcs-bucket/sample_data.csv",
    header=True,
    inferSchema=True
)
```

Make sure you've configured GCS credentials on your cluster before running this.

Step 4: Transform Data

Apply a simple filter and select specific columns: `transformed_df = df.filter(df.amount > 100).select("id", "amount")`

- Keeps only rows where `amount > 100`.
- Selects only `id` and `amount` columns for output.

Step 5: Write Transformed Data

Write results back to DBFS in Parquet format:
transformed_df.write.mode("overwrite").parquet("/FileStore/tables/etl_output")

- `mode("overwrite")` ensures previous output is replaced.
- **Parquet format** is chosen for efficiency and columnar storage.

Step 6: Verify the Transformation

Preview the transformed data directly in the notebook: `transformed_df.show()`. This displays the first 20 rows so you can confirm your ETL worked as expected.

Step 7: Execute the Notebook

- To run each cell: **Shift + Enter**.
- Databricks will send the code to the Spark cluster, which executes it in parallel.

Verification Checklist

- Notebook created and attached to cluster.
- Data successfully read from DBFS or GCS.
- Transformation applied (rows filtered by `amount > 100`).
- Output written to `/FileStore/tables/etl_output`.
- Results previewed with `.show()`.

Summary

In this lab, you:

- Created a Databricks notebook and Spark session.
- Read data from DBFS (or GCS).
- Transformed data with filtering and column selection.
- Saved results in Parquet format.
- Verified transformations using `.show()`.

This **read → transform → write** pattern is a core workflow in PySpark ETL.